

Improved Low-Overhead Communication-Efficient String Reconciliation and Edit Distance

Michael T. Goodrich¹[0000-0002-8943-191X], Gonzalo Navarro²[0000-0002-2286-741X], and Claire A. To¹[0009-0008-9102-2219]

¹ University of California, Irvine, Irvine CA, USA

² Dept. of Computer Science, University of Chile, Santiago, Chile

Abstract. Suppose two parties, Alice and Bob, hold long character strings, X and Y , respectively, and they are interested in determining how similar X and Y are. Moreover, they want to exchange the strings with cost proportional to their degree of dissimilarity. Such problems arise, for example, in database and file system synchronization operations, as well as in DNA sequence comparisons. Since the strings are long, we are interested in methods that are communication-efficient and have low overhead in terms of the computations that Alice and Bob must perform, when the strings are similar enough. In this paper, we provide a simple low-overhead communication-efficient algorithms for such string reconciliation and edit distance problems, determining the edit distance k between X and Y using only $O(k \log^3 n)$ bits of communication and $O(n \log k)$ time overhead, with high probability.

Keywords: distributed algorithms · randomized algorithms · edit distance · hash functions · communication complexity · string reconciliation

1 Introduction

A fundamental challenge in distributed computing lies in managing distributed data across communication networks. A problem arising frequently is that of checking or maintaining consistency across records in replicated databases, synchronizing files in peer-to-peer systems, and repairing or updating databases across the network. Thus, there is a need for efficient distributed protocols for reconciling character strings, which can, e.g., represent files, Web pages, or DNA sequences, avoiding the communication overhead of retransmitting the whole strings. As a concrete scenario, sequencing labs repeatedly generate genomes, and their clients then download them over the Internet to update genome libraries. In such scenarios, where clients already hold other genomes of the same species, which are known to be very similar to the new ones, clients and servers would like to minimize communication complexities as well as computational overheads.

In some cases, version control systems can keep track of where the differences are, or both parties can share a common string, so that they can easily exchange only the needed edits to reconcile the strings. The scenarios we have described,

however, may lack such shared string or edit information, so the parties *know* that the differences between their strings are small, but do not know *which* they are. Still, they want to find those differences, and reconcile the strings, within a communication cost that depends only on the amount of differences.

1.1 Problem Statement

Let $X = x_1x_2\dots x_l$ and $Y = y_1y_2\dots y_n$ be two strings held by two parties, Alice and Bob, respectively, where each $x_i, y_j \in \Sigma$, for $i = 1, 2, \dots, l$ and $j = 1, 2, \dots, n$, and Σ is an alphabet whose size may or may not depend on l or n . In the case where $l = n$, the *Hamming distance*, $\text{hd}(X, Y)$, between X and Y is the number of indices, i , where $x_i \neq y_i$, that is, the number of character substitutions needed to transform X into Y [8, 23]. A related notion that does not assume $l = n$ is the ***edit distance***, also known as *Levenshtein distance*, $L(X, Y)$, between X and Y : it is the minimum number character insertions, deletions, or substitutions needed to transform X into Y (or vice versa).

In this paper, we are interested in distributed string reconciliation algorithms with optimal (i.e., linear) computational overhead and small communication complexity for computing the edit distance between the two strings, X and Y , that are respectively held by Alice and Bob. This turns out to be equivalent to the problem of Alice and Bob exchanging their strings X and Y with communication cost based on their communicating a minimal set of edits between X and Y . We assume that Alice and Bob can communicate over some channel but they do not share any other computational resources, such as CPUs or memory. The ***communication complexity*** for a communication protocol for Alice and Bob is the number of bits that are communicated between them over the course of their distributed protocol, where we assume initially that neither party has knowledge of the other party's string.

Our results will hold ***with high probability*** (whp), meaning that they occur with probability $1 - 1/n^c$, for some constant $c \geq 1$. W.l.o.g. let us assume $l \leq n$. Our protocols use various independent sources of randomness, e.g., perfect hash functions, block boundary detection, chunk checksums, and IBLT hashing, each of which satisfies the whp guarantee.

1.2 Our Contributions

We present a simple algorithm that achieves a communication complexity of $O(k \log^3 n)$ bits and $O(n \log k)$ time overhead, where k is the edit distance, with a probability of success over $1 - 1/n^c$ for any constant c .

Our method involves a non-trivial use of the invertible Bloom lookup table (IBLT) data structure [9, 10, 12]. This is an improvement on the IBLT-based technique introduced in our recent paper [13], with which we share all the introductory sections for self-containedness.

2 Related Work

There has been considerable prior work on the general distributed edit distance problem, but we are not aware of any previous methods that are simple and communication-efficient while also having low computational overhead, which refers to the total number of local RAM operations performed by Alice and Bob, excluding time spent transmitting bits over the communication channel. For example, Orlitsky [25] gives a communication-optimal method that achieves $O(k \log n)$ communication complexity, where k is an upper bound on the edit distance between two strings of size $\Theta(n)$, but requires computational overhead that is $n^{O(k)}$, i.e., exponential in k . The computational overhead for such protocols has been subsequently improved [3, 5, 14, 15], but the best methods still require $O(n \text{polylog}(n))$ overhead and are still fairly complicated. For instance, Chakraborty, Goldenberg, and Koucký [5] present a protocol with communication efficiency of $O(k^2 \log n)$ bits and $O(n \text{polylog}(n))$ overhead by embedding strings in a Hamming space using random walks and doing their communication in this Hamming space. Belazzougui and Zhang [4] achieve a communication complexity of $O(k(\log^2 k + \log n))$ bits and $O(n \text{polylog}(n))$ overhead assuming $k < n^{1/c}$ for a large constant c , but their method is quite complex and their bounds have large constant factors. In general, the previous methods for general distributed edit distance [3–5, 14, 15] all require $O(n \text{polylog}(n))$ overhead, or worse, which in practice is unacceptable for the long strings that arise in various real-world settings.

For specialized string reconciliation, Agarwal, Chauhan, and Trachtenberg [1] present distributed methods for reconciling typical real-world strings. Their method has communication complexity that scales linearly in the edit distance between the reconciling strings, as is also true for our methods. Their reconstruction methods are based on using masks and shingling to encode the strings and enumerating Eulerian paths to perform the reconciliation. As they admit, however, the number of such paths (and, hence, the computational burden of such algorithms) can be exponentially large, hence, they do not achieve low overhead. Like our approach, Kontorovich and Trachtenberg [19] reduce string reconciliation to the set reconciliation problem, but their approach still has suboptimal performance. Song and Trachtenberg [26] revisit the approach using masks and shingles with an improved Eulerian-path reconstruction method, which they show empirically improves over the prior work by Agarwal, Chauhan, and Trachtenberg [1], but the asymptotic overhead of their method is unfortunately still not efficient.

3 Invertible Bloom Lookup Table (IBLT)

IBLT [9, 12] is a space-efficient probabilistic data structure, based on Bloom filters, that supports various operations implemented by the XOR primitive. It can be used to solve the *set reconciliation* problem [10], which is the task of synchronizing two sets stored at two nodes across a communication link. IBLTs

allow the parties to efficiently identify the items that are unique to each set so that both parties can update their local copies and obtain the other set as well. The main step in this process is to compute a *set difference*, finding the set of elements that one party possesses but the other does not.

Suppose we are given a set $S = \{s_1, s_2, \dots, s_n\}$, which we want to store probabilistically in an IBLT table, T , with m cells. We assume that we have an **encoding function**, `encode`, which encodes each element, s_i , into an associated unique, fixed-length binary string key, e_i . We also have λ hash functions that map any key to λ distinct locations, and we let $\text{HashToIndices}(e_i, \lambda, m)$ denote the set of λ locations determined by these hash functions for the key e_i . Each IBLT cell, $T[j]$, stores a (**keysum**, **hashsum**) pair such that **keysum** contains the bitwise-XOR (denoted $\oplus$) of all encoded keys e_i that are mapped to $T[j]$ by one of T 's hash functions; **hashsum** contains the XOR of all fingerprints or hashes $H(e_i)$, where $H(e_i)$ is a b -bit secondary hash of e_i , such as a checksum, or cryptographic hash, used to verify or detect errors in the decoding process.

The IBLT supports both insertion and deletion of elements using XOR operations. Inserting an element corresponds to XOR-ing its encoded value into the table (at the λ locations), while deleting it corresponds to *the same* operations. To encode an entire set, S , we simply insert each of its elements.

For a set reconciliation protocol, Alice and Bob build respective tables, T_A and T_B , by inserting all the elements of their sets, A and B , respectively, into initially zeroed IBLTs of the same size using the same hash functions, and exchange the tables. From these two IBLTs, T_A and T_B , both Alice and Bob can compute an IBLT representing the symmetric difference between A and B . This simple method is shown in Algorithm 1. The fact that this algorithm computes an IBLT representation of the symmetric difference of A and B follows immediately from the fact that $x \oplus x = 0$ for any value x .

Listing Set Entries If an IBLT is not “too” full, we can list out its entries. The listing of entries or decoding is a destructive $O(m)$ -time procedure that recovers all keys in the IBLT or reports that the IBLT is too full to achieve this successfully. Say that a cell $T[i]$ is **pure** if

$$H(T[i].\text{keysum}) = T[i].\text{hashsum},$$

which clearly holds for a cell holding exactly one key but could admittedly also hold if there is a checksum collision. Note that if a cell is pure, we can recover its key by extracting the value from $T[i].\text{keysum}$ and then deleting the element from the table, which is also referred to as **peeling** the key from the corresponding

Algorithm 1 IBLT Subtract ($T = T_A \oplus T_B$)

```

1: for  $i = 0$  to  $m - 1$  do
2:    $T[i].\text{keysum} \leftarrow T_A[i].\text{keysum} \oplus T_B[i].\text{keysum}$ 
3:    $T[i].\text{hashsum} \leftarrow T_A[i].\text{hashsum} \oplus T_B[i].\text{hashsum}$ 

```

Algorithm 2 IBLT Decode T

```
1: while  $\exists i$ , s.t.  $H(T[i].\text{keysum}) = T[i].\text{hashsum}$  do
2:    $s \leftarrow \text{decode}(T[i].\text{keysum})$ 
3:   Output  $s$ 
4:    $\text{delete}(T, s)$ 
5: for  $i = 0$  to  $m - 1$  do
6:   if  $T[j].\text{keysum} \neq 0$  OR  $T[j].\text{hashsum} \neq 0$  then
7:     return FAIL
8: return SUCCESS
```

λ hashed cells. This may make other cells pure, which can eventually allow us to recover all the values stored in the IBLT. Algorithm 2 shows this process.

Lemma 1. *The probability that a cell containing $t \geq 2$ keys produces a false positive on the purity test is 2^{-b} .*

Proof: Let H be a uniform random hash function that produces b -bit hash values. Suppose a cell contains $t \geq 2$ keys, with $\text{keysum} = x_1 \oplus x_2 \oplus \dots \oplus x_t$ and $\text{hashsum} = H(x_1) \oplus H(x_2) \oplus \dots \oplus H(x_t)$. Define a random variable $Z = H(\text{keysum}) \oplus \text{hashsum} = H(x_1 \oplus \dots \oplus x_t) \oplus H(x_1) \oplus \dots \oplus H(x_t)$. Because H is uniform and random, $H(x_1 \oplus \dots \oplus x_t)$ is independent of $H(x_1), \dots, H(x_t)$ and each value is uniformly distributed over $\{0, 1\}^b$. Thus, the probability of failure is 2^{-b} . ■

Corollary 1. *In an IBLT of m cells, the probability that the purity test yields a false positive for any cell is at most $m2^{-b}$.*

Proof: Apply the union bound to the failure probability from Lemma 1. ■

In our setting, we want to recover a symmetric difference of size at most d with high probability, even if d is a constant. We use an IBLT of size $m = c \cdot d \log n$ for a $c \geq 1$, and λ being $\Theta(\log n)$, so that by an analysis similar to that of Goodrich, Kitagawa, and Mitzenmacher [11], when the current number of elements in the IBLT is at most d , we can successfully list all entries of T whp:

Theorem 1 *Let T be an IBLT with $m = cd \log n$ cells and $\lambda = (c/2) \log n$ hash functions, for $c \geq 4$. If T stores at most d elements, then it can be successfully decoded by the peeling algorithm with probability at least $1 - 1/n^{c/2-1}$.*

Proof: Let X_i be a random variable that is 1 if the i -th element has no pure cell and is 0 otherwise. Since there are at most d elements in T and it has $m = cd \log n$ cells and $\lambda = (c/2) \log n$ hash functions, at least half the cells in the IBLT are empty. Thus, any one of the λ hash functions will map an element to what would otherwise be an empty cell with probability at least $1/2$. Since the λ hash functions are independent, this implies that

$$\Pr(X_i = 1) \leq \frac{1}{2^\lambda} = \frac{1}{n^{c/2}}.$$

Thus, by a union bound, the probability that every element in T has a pure cell is at least $1 - 1/n^{c/2-1}$. ■

To sum up, suppose two sets, A and B , of size $O(n)$, are held by Alice and Bob respectively, which are assumed to differ by at most d elements. Each party independently constructs an IBLT, T_A and T_B of $m = O(d \log n)$ cells, and inserts all elements into the table. Even if either T_A and T_B is too full to decode individually, there will be pure cells in $T_A \oplus T_B$, i.e., containing only one key. That is, with properly chosen parameters, Alice and Bob can reconcile their sets whp, in $O(n)$ time and $O(d\beta \log n)$ bits of communication, where $\beta > 0$ is the size of the keys, by exchanging their tables T_A and T_B and performing the decoding algorithm on $T_A \oplus T_B$.

4 Our Algorithm

Locally consistent parsing (LCP) [2, 7, 22] divides a string, $X = x_1 x_2 \dots x_n$, into a sequence, $\mathcal{C}$, of contiguous substrings, called **chunks**, $C_1, C_2, \dots, C_r$, so that $X = C_1 || C_2 || \dots || C_r$, where $||$ denotes concatenation. The version of LCP we use [6, 22] achieves this by defining a kind of random permutation π on the alphabet symbols of X , and dividing the string, X , by a simple algorithm that ends a chunk wherever a *local minimum* occurs. Previously to this chunking, the technique groups *runs* of equal symbols. Precisely

1. Every maximal sequence of two or more equal consecutive symbols, $x_i x_{i+1} \dots x_j = a^{j-i+1}$, for $j > i$, is replaced in X by a new symbol $\langle a, j - i + 1 \rangle$. Let $X' = x'_1 x'_2 \dots x'_{n'}$ be the resulting sequence.
2. A random injective function π is defined from the alphabet of X' to integers in $[0, n^c - 1]$ for some constant $c \geq 2$. Crucially, $\pi(\langle a, k \rangle) = \pi(a)$ for any k . Also, for technical convenience, we fix $\pi(x'_1) = n^c - 1$ and $\pi(x'_{n'}) = 0$.
3. The string, X' , is divided into a sequence of chunks: a chunk ends at every position i such that $\pi(x'_{i-1}) > \pi(x'_i) < \pi(x'_{i+1})$; the last chunk ends at position n' .
4. The resulting sequence of chunks is called $\text{LCP}(X)$.

Note that, since there cannot be two consecutive runs or symbols with the same symbol c , it holds $\pi(x'_i) \neq \pi(x'_{i+1})$ for every i . Further, there cannot be chunks of length 1, as we show next.

Lemma 2. *Every chunk is of length at least 2.*

Proof: Follows from the fact that there cannot be two consecutive minima by definition, and that by our choice of $\pi(x'_1)$ and $\pi(x'_{n'})$, the first and last chunks cannot be of length 1 either. ■

The next lemma shows, on the other hand, that the chunks are short whp.

Lemma 3. *With high probability, the longest chunk of X' is of length $O(\log |X'|)$.*

Proof: Consider the distance between two local minima, i and j , in $\pi(X')$. The values of π in this range must have an increasing phase starting at i , of length ℓ^+ , and a decreasing phase ending at j , of length ℓ^- . Since there cannot be repeated symbols $X'[i]$ in the increasing nor in the decreasing phase, the values of π within each phase are independent. Therefore, the probability of lengths ℓ^+ or ℓ^- exceeding d is $2/2^d$. By the union bound, since there are at most $|X'|/2$ chunks in X' , the probability of some chunk exceeding length $2d$ is at most $|X'|/2^d \leq n/2^d$. By choosing $d \geq c \log_2 n$, this is at most $1/n^c$. ■

We implement π as a perfect hash function chosen uniformly at random from a 2-universal family, such as $\mathcal{H} = \{h_{\alpha,\beta}(x) = 1 + [((\alpha x + \beta) \bmod p) \bmod (n^c - 1)], \alpha, \beta \in \mathbb{N}, \alpha > 0\}$ for a prime $n^c < p = O(n^c)$ (fixing $\pi(x'_1) = 0$ and $\pi(x'_{n'}) = n^c - 1$). Any random choice $\pi = h_{\alpha,\beta}$ is perfect with probability $\Theta(1/n^{c-1})$, so we find a suitable π in $O(1)$ attempts whp, and π has an $O(\log n)$ -bit description. Further, we iterate in the quest for π until the longest chunk it produces is of length at most $2c \log_2 n$. Overall, we build π in $O(n)$ time whp.

The general process involves consecutive *rounds* of LCP: calling $X_0 = X$ the original sequence, we apply again LCP on the sequence $X_1 = \text{LCP}(X_0)$, $X_2 = \text{LCP}(X_1)$, and so on until reaching some X_r such that $|X_h| = 1$. Precisely:

1. We produce $Y_0 = \text{LCP}(X_0)$ as described above, so Y_0 is a sequence of chunks. Let π_0 be the function used for chunking; chunks are of logarithmic length.
2. We give *integer names* to the distinct chunks in Y_0 . Those names are given with a $\Theta(\log n)$ -bit hash function h_0 that is perfect for the chunks in Y_0 , so $N = h_0(C)$ is the name of chunk C .
3. The string X_1 is formed by replacing its chunks in Y_0 by their names.
4. We repeat the process, forming $Y_1 = \text{LCP}(X_1)$ with a new random function π_1 and hash function h_1 , and so on.

The names will be recorded, for all the rounds i , in sets

$$A_i = \{N \rightarrow C, C \text{ is a chunk of } Y_i \text{ assigned name } N = h_i(C)\}.$$

We now show that all Alice needs to send Bob so he can reconstruct X are the sets A_i and the hash functions π_i and h_i .

Lemma 4. *Given the sets A_i , the hash functions π_i and h_i , the integer r , and the string X_r of length 1, we can reconstruct X .*

Proof: We proceed by induction, showing that we can reconstruct every X_i , from $i = r$ to $i = 0$, where we finish because $X_0 = X$. For the base case, X_r is given. Now, assume we have decoded X_{i+1} . Using A_i , we replace every symbol $X_{i+1}[j] = N$ by the chunk C , where $N \rightarrow C$ appears in A_i . Further, any symbol of the form $\langle c, r \rangle$ in C is rewritten as r copies of c . The result is X_i . ■

Finally, we show that the total size of the sets is linear.

Lemma 5. *The total size of all the sets A_i and hash functions π_i and h_i is $O(n)$, and there are $r \leq \log_2 n$ levels.*

Proof: Collapsing the runs of X_i to form X'_i cannot increase its length, so $|X'_i| \leq |X_i|$. Further, Lemma 2 implies $|X_{i+1}| \leq |X'_i|/2$. Therefore, we obtain $|X_r| = 1$ after $r \leq \log_2 n$ rounds. There are then $O(\log n)$ functions π_i and h_i .

By connecting the names created at level i with the places where they are used in chunks of level $i + 1$, the sets A_i can be regarded as a tree with root $X_r[1]$ and leaves forming X (concretely, the union of the sets A_i can be seen as the rules of a context-free grammar that generates X). Since there are no unary nodes in this tree, it has $O(n)$ nodes. The sets A_i add up to less because they represent each name only once, thus $\sum_{i=0}^r |A_i| = O(n)$. ■

In particular, this implies that the CPU cost to reconstruct the sets is $O(n)$, in addition to the $O(n)$ time needed whp to build suitable hashes. We now state the key result that allows us using LCP for string reconciliation.

Lemma 6. *A single edit in X can alter at most $2 \log_2 n$ elements in the corresponding sets A_i .*

Proof: We prove in Appendix A that an edit operation in X_0 cannot propagate more than two altered chunks in any of the sequences X_i . Therefore, no more than $2r \leq 2 \log_2 n$ rules can be altered. ■

Let us assume that we have an upper-bound estimate, $k \geq 1$, for the edit distance between the strings, X and Y , held respectively by Alice and Bob (we later lift this assumption). Let us focus on Bob obtaining the string X from Alice; the process for Alice obtaining Bob's Y is analogous.

We first obtain common functions π_i and h_i for Alice and Bob. This can be achieved asymmetrically, as follows: Alice generates suitable functions and sends them to Bob, using only $O(\log^2 n)$ bits of communication. Bob checks that the functions are suitable for the construction of his own sets B_i from Y .³ If not, he requests a new set of functions to Alice. Only $O(1)$ iterations suffice whp. Note that, because these functions are shared, equal chunks C will receive equal names $N = h_i(C)$ and functions π_i produce the same cuts, so Lemma 6 implies that there will be at most $2k$ differing elements between the sets A_i and B_i .

Alice creates an IBLT of appropriate size to allow Bob obtaining the elements of each set A_i with probability at least $1 - 1/\eta$, for our desired confidence parameter of successful decoding, $n < \eta \leq n^c$. Since there will be at most $2k$ differing elements between A_i and B_i , Alice builds IBLTs of $O(k \log \eta)$ cells, with each cell storing a **keysum** and **hashsum** of $\Theta(\log n)$ bits. Alice then transmits r , X_r , and r IBLTs encoding her sets A_i , from A_{r-1} to A_0 , in rounds.

Bob will proceed by rounds, obtaining the strings X_i from X_r to $X_0 = X$, by following Lemma 4. The string X_r is sent explicitly by Alice, which gives the base of the induction. In general, Bob has already decoded X_{i+1} and wants to decode X_i from X_{i+1} and A_i . Since there are at most $2k$ chunk contents C that differ between A_i and B_i , the IBLT will allow Bob recovering A_i from B_i whp. Precisely, Bob will identify, whp, the at most $2k$ elements in $A_i \setminus B_i$,

³ To make them compatible, we take $n = \max(|X|, |Y|)$ when building the functions, and the domain of the injective functions is sufficient to map any integer name.

which he will request from Alice; she will send him back the explicit chunks. Each such chunk requires $O(\log^2 n)$ bits of communication. With X_{i+1} and A_i , Bob will then reconstruct X_i using Lemma 4. The total failure probability is $O((1/\eta) \log n)$, so we still succeed whp.

The r IBLTs add up to $O(k \log n \log \eta)$ cells, and each cell requires $O(\log n)$ bits because the chunks are of constant size and the alphabet and names requires $O(\log n)$ bits each. On the other hand, the exchanged chunks add up to $O(k \log n)$, and each requires $O(\log^2 n)$ bits of communication. Therefore, we have the following result, which yields $O(k \log^3 n)$ -bit communication complexity in order to succeed whp.

Theorem 2 *Let k be an a-priori-known upper bound on the edit distance between two strings, X and Y , of length $\Theta(n)$ and defined over an alphabet of size $O(n)$, held by two separate parties, Alice and Bob. Then our IBLT-chunking algorithm allows Alice and Bob to determine the other party's string in $O(n)$ time using*

$$O(k \log^2 n (\log n + \log \eta))$$

bits of communication, all with probability at least $1 - 1/\eta$ for any $n < \eta \leq n^c$ for any desired constant $c \geq 1$.

Once both Alice and Bob have the chunks of both strings, they can compute their edit distance in $O(n + k^2)$ additional time [20].

Thus, when $k = O(\sqrt{n})$, we can also compute the precise edit distance within this cost, which may outperform our general fast-sliding algorithm. To conclude, we note that we can obtain a similar result *without* knowing an *a priori* bound k on the edit distance, by slightly increasing the time overhead.

Corollary 2. *The same result of Theorem 2 can be obtained if k is the actual edit distance between X and Y , a priori unknown, with $O(n \log k)$ time overhead.*

Proof: Apply Theorem 2 with upper bound $k' = 1, 2, 4, 8, \dots$ until it succeeds. With probability at least $1 - 1/\eta$, the try with $k \leq k' < 2k$ will succeed, and each party will obtain the other's string. The total communication cost is as in Theorem 2, replacing k by $\sum_{j=0}^{\lceil \log_2 k \rceil} 2^j = O(k)$, and the time overhead is $O(n \log k)$ because we process the chunks $O(\log k)$ times, once per value of k' . ■

5 Conclusions

We have introduced a new IBLT-based communication efficient algorithm for string reconciliation between two parties that separately store two strings of size $\Theta(n)$, improving our recent result also based on IBLT [13] by exchanging only $O(k \log^3 n)$ bits whp.

Our technique is based on generating a randomized grammar that outputs the string, and exchanging the grammars. We show that each edit alters only $O(\log n)$ nonterminals in the grammar, if a specific grammar construction [6]

is used. Our technique has the additional benefit that the grammar it uses is guaranteed to generate a representation of size $O(\gamma \log(n/\gamma))$, where γ is the size of the smallest attractor [16] of the string to exchange. For example, γ lower-bounds the size of the Lempel-Ziv parse [21] of the string. If the texts Alice and Bob have to exchange are very repetitive (e.g., large collections of genomes of the same species, as in genomic databases), their γ value is much less than their collection size n [24]. In such cases, they may have the texts directly compressed in grammar form, and can reconcile their databases without spending time or space proportional to n , but to γ . By using a slightly more complex grammar [17], we can obtain the same result with a grammar of size $O(\delta \log(n/\delta))$, where $\delta \leq \gamma$ is the normalized substring complexity of the strings [18].

References

1. S. Agarwal, V. Chauhan, and A. Trachtenberg, “Bandwidth efficient string reconciliation using puzzles,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 17, no. 11, pp. 1217–1225, 2006.
2. T. Batu and S. C. Sahinalp, “Locally consistent parsing and applications to approximate string comparisons,” in *9th International Conference on Developments in Language Theory*, 2005, pp. 22–35.
3. D. Belazzougui, “Efficient deterministic single round document exchange for edit distance,” *CoRR*, vol. 1511.09229, 2015.
4. D. Belazzougui and Q. Zhang, “Edit distance: Sketching, streaming, and document exchange,” in *Proc. IEEE Symp. on Foundations of Computer Science (FOCS)*, 2016, pp. 51–60.
5. D. Chakraborty, E. Goldenberg, and M. Koucký, “Streaming algorithms for embedding and computing edit distance in the low distance regime,” in *Proc. ACM Symp. on Theory of Computing (STOC)*, 2016, pp. 712–725.
6. A. R. Christiansen, M. B. Ettienne, T. Kociumaka, G. Navarro, and N. Prezza, “Optimal-time dictionary-compressed indexes,” *ACM Transactions on Algorithms*, vol. 17, no. 1, p. article 8, 2020.
7. R. Cole and U. Vishkin, “Deterministic coin tossing and accelerating cascades: Micro and macro techniques for designing parallel algorithms,” in *18th Annual ACM Symposium on Theory of Computing (STOC)*, 1986, pp. 206–219.
8. M. Crochemore, C. Hancart, and T. Lecroq, *Algorithms on Strings*. Cambridge University Press, 2007.
9. D. Eppstein and M. T. Goodrich, “Straggler identification in round-trip data streams via newton’s identities and invertible Bloom filters,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 23, no. 2, pp. 297–306, 2010.
10. D. Eppstein, M. T. Goodrich, F. Uyeda, and G. Varghese, “What’s the difference? Efficient set reconciliation without prior context,” *SIGCOMM Computer Communication Review*, vol. 41, no. 4, p. 218–229, Aug. 2011.
11. M. T. Goodrich, R. Kitagawa, and M. Mitzenmacher, “Parallel peeling of invertible Bloom lookup tables in a constant number of rounds,” in *Proc. Int. Conf. on Current Trends in Theory and Practice of Computer Science*, 2025, pp. 70–84.
12. M. T. Goodrich and M. Mitzenmacher, “Invertible Bloom lookup tables,” *CoRR*, vol. 1101.2245, 2015.
13. M. T. Goodrich, G. Navarro, and C. A. To, “Simple low-overhead communication-efficient string reconciliation and edit distance,” *CoRR*, vol. 2608.19149, 2026.

14. U. Irmak, S. Mihaylov, and T. Suel, “Improved single-round protocols for remote file synchronization,” in *Proc. 24th Joint Conf. of the IEEE Computer and Communications Societies*, vol. 3, 2005, pp. 1665–1676.
15. H. Jowhari, “Efficient communication protocols for deciding edit distance,” in *Proc. European Symp. on Algorithms (ESA)*, 2012, pp. 648–658.
16. D. Kempa and N. Prezza, “At the roots of dictionary compression: String attractors,” in *Proc. 50th Annual ACM Symposium on the Theory of Computing (STOC)*, 2018, pp. 827–840.
17. T. Kociumaka, G. Navarro, and F. Olivares, “Near-optimal search time in δ -optimal space, and vice versa,” *Algorithmica*, vol. 86, pp. 1031–1056, 2024.
18. T. Kociumaka, G. Navarro, and N. Prezza, “Toward a definitive compressibility measure for repetitive sequences,” *IEEE Transactions on Information Theory*, vol. 69, no. 4, pp. 2074–2092, 2023.
19. A. Kontorovich and A. Trachtenberg, “String reconciliation with unknown edit distance,” in *Proc. IEEE Int. Symp. on Information Theory*, 2012, pp. 2751–2755.
20. G. M. Landau, E. W. Myers, and J. P. Schmidt, “Incremental string comparison,” *SIAM Journal on Computing*, vol. 27, no. 2, pp. 557–582, 1998.
21. A. Lempel and J. Ziv, “On the complexity of finite sequences,” *IEEE Transactions on Information Theory*, vol. 22, no. 1, pp. 75–81, 1976.
22. K. Mehlhorn, R. Sundar, and C. Urig, “Maintaining dynamic sequences under equality tests in polylogarithmic time,” *Algorithmica*, vol. 17, no. 2, pp. 183–198, 1997.
23. G. Navarro, “A guided tour to approximate string matching,” *ACM Computing Surveys*, vol. 33, no. 1, pp. 31–88, 2001.
24. —, “Indexing highly repetitive string collections, part I: Repetitiveness measures,” *ACM Computing Surveys*, vol. 54, no. 2, p. article 29, 2021.
25. A. Orlitsky, “Interactive communication: Balanced distributions, correlated files, and average-case complexity,” in *Proc. IEEE Symp. on Foundations of Computer Science (FOCS)*, 1991, pp. 228–238.
26. B. Song and A. Trachtenberg, “Scalable string reconciliation by recursive content-dependent shingling,” in *Proc. 57th Allerton Conf. on Communication, Control, and Computing (Allerton)*, 2019, pp. 623–630.

A Maximum Number of Affected Chunks

We prove that a single change in the base sequence X_0 (i.e., an insertion, a deletion, or a substitution of an integer) cannot induce more than 2 changes in the structure of any subsequent sequence X_i . We proceed by induction on i .

Lemma 7 (base case). *Any single insertion, deletion, or substitution in X_0 induces at most 2 consecutive changes in X_1 .*

Proof: Let us first consider a sequence X_0 with no equal consecutive elements. We map the relations between adjacent elements to a sequence of signs $S_0 = s_1, s_2, \dots, s_{n-1}$, where $s_j = +$ if $x_j < x_{j+1}$ and $s_j = -$ if $x_j > x_{j+1}$. Under this representation, every local minimum (which marks a chunk endpoint) corresponds uniquely to the occurrence of the substring $-+$ in S_0 . We analyze the three edit operations on X_0 and their impact on S_0 :

Insertions: Inserting an element at index j splits the original sign s_j into a pair $s's''$. Depending on the value of the inserted element relative to its new neighbors, a $+$ can transition into $++$, $+-$, or $-+$, while a $-$ can transition into $--$, $+-$, or $-+$. Because this operation only increases the local count of $-+$ patterns by at most one, it alters at most one existing chunk boundary and inserts at most one new chunk boundary.

Deletions: Deleting x_j removes the signs s_{j-1} and s_j , replacing them with a single new sign s' that represents the relation between x_{j-1} and x_{j+1} . Thus, pairs like $++$ or $--$ contract to $+$ or $-$, while $+-$ or $-+$ collapse into either $+$ or $-$. This contraction can destroy at most one $-+$ pattern, altering at most one chunk and deleting at most one chunk boundary.

Substitutions: Modifying x_j alters its relation with both adjacent neighbors, changing the pair $s_{j-1}s_j$ into a new pair $s's''$. Case-wise analysis shows that a substitution cannot create two consecutive local minima ($-+-$) from a non-minimum configuration of $s_{j-1}s_j$ (i.e., $++$, $--$, or $+-$). Thus, a substitution can introduce at most one new $-+$ pattern or destroy at most one existing $-+$ pattern, modifying at most two adjacent chunks.

Let us now consider an edit that occurs within or adjacent to a run $\langle a, k \rangle$. Remind that all runs of symbol a map to the same value $\pi(a)$.

Insertions: Inserting an element a inside the run $\langle a, k \rangle$ only changes that symbol in its chunk, to $\langle a, k+1 \rangle$. Inserting $d \neq a$ inside the run of $b\langle a, k \rangle c$ yields in X'_0 the sequence of mapped values $\dots, \pi(b), \pi(a), \pi(d), \pi(a), \pi(c), \dots$. This expands the local sign sequence by inserting a $+-$ or $-+$ pair. Just as in the non-run case, this introduces at most one new $-+$ pattern.

Deletions: Deleting an element inside a run only changes the run symbol in its chunk. However, if the deletion removes a separating element b in the configuration $\langle a, k_1 \rangle b \langle a, k_2 \rangle$, the two runs fuse into a single run $\langle a, k_1 + k_2 \rangle$. This fusion removes the corresponding $+-$ or $-+$ transitions from the sign sequence, destroying a chunk boundary rather than creating one.

Substitutions: A substitution inside a run acts as an insertion of a new distinct symbol, matching the insertion bounds. If, instead, a substitution changes a separating element b into a within the context $\langle a, k_1 \rangle b \langle a, k_2 \rangle$, it triggers a run fusion identical to the deletion case, removing a chunk boundary.

In all scenarios, the net modification to the sequence of chunk boundaries is bounded by ± 1 , altering or creating at most 2 consecutive chunks in X_1 . ■

Lemma 8 (inductive step). *If the sequence X_i undergoes at most 2 consecutive changes, the sequence X_{i+1} also undergoes at most 2 consecutive changes.*

Proof: Let S_i be the sign sequence for X_i . Two consecutive changes in X_i produce at most two adjacent or overlapping modifications in a window of S_i :

1. If the 2 changes are deletions or substitutions, they rewrite a window of at most 3 consecutive signs: $s_{j-1}s_js_{j+1} \rightarrow s's''s'''$.

2. If only one change is an insertion, the operations rewrite a window of two consecutive signs into at most 3: $s_{j-1}s_j \rightarrow s's''s'''$.
3. If the 2 changes are insertions, they split a single original sign into 3 new internal signs: $s_j \rightarrow s's''s'''$.

When the edits affect a run $\langle a, k \rangle$, they may replace it by four new symbols, like $\langle a, k_1 \rangle bc \langle a, k_2 \rangle$. Still, at most 3 signs are added: if the run is surrounded by symbols $s_{j-1}s_j$, then the insertion/substitution of bc introduces new signs $s's''s'''$ between s_{j-1} and s_j . Note s_{j-1} and s_j do not change because all the runs with letter a are regarded as equal by π_i . Deletions only change k_1 or k_2 .

To bound the largest disruption, we observe the *extended* sign window, which encompasses the modified signs plus 1 unchanged original sign to the immediate left and 1 to the right. The 3 new internal signs, combined with these 2 unchanged boundary signs, form an extended window of exactly 5 signs in total.

A string of 5 signs can accommodate at most $\lfloor 5/2 \rfloor = 2$ occurrences of the chunk endpoint pattern $-+$. Because all signs outside this extended 5-sign window remain unaltered, no additional chunk boundaries can be created or displaced.

Thus, the total number of added, removed, or shifted chunk boundaries in X_{i+1} is at most 2. Since each boundary maps directly to an element in X_{i+1} , the subsequent layer undergoes at most 2 consecutive changes. ■